

File I/O

Day 9 — getting real data in, and results out



Boston University

90 min

Concepts + live code

30 min

Practice notebook

1

Keyword: with

Before we start — five questions from Day 8

1 What must a child `__init__` call, and when?

→ `super().__init__(...)` — first, before its own setup.

2 Overriding vs extending?

→ Overriding replaces; `super()` keeps the parent's work.

3 Is `isinstance(student, Person)` True?

→ Yes — a Student really is a Person.

4 Does polymorphism need inheritance?

→ No. Duck typing needs only a shared method name.

5 IS A or HAS A?

→ IS A → inherit. HAS A → composition.

OOP is done. Today we stop typing data into the notebook and start reading it from files.

How today's two hours are structured

1

Reading text files

25 min

`open()`, `with`, and four ways to read

2

Writing & file modes

20 min

`r` · `w` · `a` · `x`, and the one that deletes

3

CSV files

25 min

Why `split(',')` is not enough

4

Paths & errors

10 min

`FileNotFoundError`, encoding, `pathlib`

5

Mini build

10 min

Read a CSV, process it, write a report

6

Practice notebook

30 min

You code, I walk around

Two habits to leave with today: always use `with`, and never open a real file in "w" mode by accident.

By the end of today you will be able to

1

Read

Open a file safely and get its contents, line by line

2

Write

Create and append to files, and pick the right mode every time

3

Handle CSV

Read and write real comma-separated data without breaking on commas

4

Survive

Cope with a missing file instead of crashing

This is the bridge to the rest of the course

In two weeks you will write `pd.read_csv("data.csv")` and get a DataFrame in one line. Today you do the same job by hand — and that is the point. When a file has an odd separator, a missing header or a strange encoding, pandas will not save you unless you understand what it was doing.

1

Reading Text Files

open, with, and four ways to read · 25 minutes

- `open()`
- The `with` statement
- `read` / `readline` / `readlines`
- Looping a file

open() — and why you always use with

The manual way

```
f = open("data.txt")
text = f.read()
f.close()           # you MUST remember this

# If read() raises, close() never runs
# and the file stays open.
```

An open file holds an operating-system handle. Leak enough of them and the program cannot open anything at all.

The way you should always write it

```
with open("data.txt") as f:
    text = f.read()

# f is closed automatically here -
# even if an error was raised inside.
```

The with block guarantees the file is closed when you leave it, however you leave it.

The full form of open()

```
open(path, mode="r", encoding="utf-8")
```

with is a general Python feature, not a file feature — you will meet it again with database connections and locks.

Always pass `encoding="utf-8"` when you open a text file. Without it Python uses whatever the machine happens to prefer, so a file that reads perfectly on your laptop can fail on someone else's. One argument removes an entire category of bug.

Four ways to read — and which to use

Method	Gives you	Use it when
<code>f.read()</code>	the whole file as ONE string	the file is small and you want it all
<code>f.readline()</code>	the next single line	you need just the header, or one line
<code>f.readlines()</code>	a list of every line	you need a list and the file is small
<code>for line in f:</code>	one line at a time, lazily	almost always — any size of file

The one to default to

```
with open("marks.txt", encoding="utf-8") as f:
    for line in f:
        print(line.strip())

# Reads one line at a time - a 10 GB file
# uses no more memory than a small one.
```

⚠ Every line keeps its newline

```
f.readlines()
-> ['Ravi\n', 'Sara\n', 'Amit']

line == "Ravi"      -> False !
line.strip() == "Ravi" -> True

# The last line often has no \n at all
```

2

Writing & Modes

The mode that deletes your file · 20 minutes

- `r · w · a · x`
- `write` vs `writelines`
- Newlines

File modes — pick the wrong one and data disappears

Mode	Means	If the file exists	If it does not
"r"	read (the default)	reads it	FileNotFoundError
"w"	write	ERASES IT COMPLETELY	creates it
"a"	append	adds to the end	creates it
"x"	exclusive create	FileExistsError	creates it
"r+"	read and write	opens without erasing	FileNotFoundError

"w" does not warn you

```
with open("important.csv", "w") as f:
    f.write("oops")

# The original contents are gone. Instantly.
```

There is no undo, no recycle bin, no confirmation.

How to protect yourself

- Use "a" when you mean to add to a file
- Use "x" when the file must not already exist — it refuses rather than overwrites
- Write to a NEW filename, never over your input

Writing to a file

write() adds NO newline

```
with open("out.txt", "w") as f:
    f.write("Ravi")
    f.write("Sara")

# The file now contains: RaviSara

    f.write("Ravi\\n")    # you add it yourself
```

Three ways to write many lines

```
for n in names:
    f.write(n + "\\n")

f.writelines(n + "\\n" for n in names)
# writelines adds NO newlines either

print(n, file=f)        # print DOES add one
```

Appending, and the pattern you will actually use

```
with open("log.txt", "a", encoding="utf-8") as f:
    f.write(f"{name} scored {mark}\\n")

# "a" keeps everything already there.
```

Two details worth knowing: write() returns the number of characters written, which is occasionally useful and usually ignored. And nothing is guaranteed to reach the disk until the file is closed — which the with block handles for you, and is one more reason to use it.

3

CSV Files

Where real data actually lives · 25 minutes

- Why `split(',')` fails
- `csv.reader`
- `csv.DictReader`
- Writing CSV

Why `line.split(",")` is not enough

It works — until it doesn't

```
name,city,mark
Ravi,Pune,88
Sara,"Mumbai, MH",91
```

<- a comma
INSIDE a field

```
line.split(",")
-> ['Sara', '"Mumbai, MH"', '91']
    four fields instead of three
```

The csv module handles it

```
import csv

with open("data.csv", newline="") as f:
    for row in csv.reader(f):
        print(row)

# ['Sara', 'Mumbai, MH', '91']
```

It understands quoted fields, escaped quotes, and embedded newlines — none of which `split()` can.

Two things to remember when opening a CSV

```
open(path, newline="", encoding="utf-8")
```

`newline=""` lets the csv module handle line endings itself; without it you can get blank rows between every record on Windows. And every value comes back as a `STRING` — “88” is text until you cast it, which is Day 1's `input()` lesson in a new place.

Two ways to read a CSV

csv.reader — each row is a LIST

```
import csv

with open("marks.csv", newline="") as f:
    reader = csv.reader(f)
    header = next(reader)      # skip the header
    for row in reader:
        print(row[0], int(row[2]))

# row = ['Ravi', 'Pune', '88']
# You address fields by POSITION.
```

csv.DictReader — each row is a DICT

```
import csv

with open("marks.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], int(row["mark"]))

# row = {'name': 'Ravi', 'city': 'Pune',
#        'mark': '88'}
# The header becomes the keys, automatically.
```

Prefer DictReader. `row["mark"]` survives someone adding a column; `row[2]` silently starts reading the wrong field. It is Day 5's argument for dictionaries over lists — the data explains itself — applied to files. DictReader also skips the header for you, which is one fewer thing to forget.

Writing a CSV

csv.writer — rows as lists

```
import csv

rows = [{"Ravi", 88}, {"Sara", 91}]

with open("out.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["name", "mark"])    # header
    w.writerows(rows)              # all at once
```

csv.DictWriter — rows as dicts

```
rows = [{"name": "Ravi", "mark": 88},
        {"name": "Sara", "mark": 91}]

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "mark"])
    w.writeheader()                # writes the header row
    w.writerows(rows)
```

The csv module quotes for you

```
w.writerow(["Sara", "Mumbai, MH"])
# file gets: Sara,"Mumbai, MH"
```

A value containing a comma is quoted automatically, so the file you produce can be read back correctly. Writing CSV by hand with commas and f-strings works right up until one of your values contains a comma — and then it quietly produces a broken file that something downstream will misread.

4

Paths & Errors

Coping when the file is not there · 10 minutes

- FileNotFoundError
- Checking first
- pathlib

When the file is not there

The errors you will actually meet

```
FileNotFoundError  wrong name or folder
PermissionError    no rights to open it
IsADirectoryError  that path is a folder
UnicodeDecodeError wrong encoding
FileExistsError    "x" mode, already there
```

Handle it — Day 1's try / except

```
try:
    with open("data.csv") as f:
        rows = f.readlines()
except FileNotFoundError:
    print("No data file – starting empty")
    rows = []
```

Checking first

```
from pathlib import Path

if Path("data.csv").exists():
    ...
```

Useful, but try/except is safer: the file could vanish between the check and the open.

Paths, and a note about Colab

A bare "data.csv" means "in the current folder". In Colab that folder empties when the session ends, so anything you write today is temporary — upload with the file panel on the left, or mount Google Drive to keep things.

Debug corner — eight file bugs

You wrote	What happens	Fix
<code>open("f.txt", "w")</code> to read	the file is erased instantly	<code>"r"</code>
<code>f = open(...)</code> no <code>close()</code>	handle leaked, data may not save	<code>with open(...) as f:</code>
<code>line == "Ravi"</code>	False — the newline is still there	<code>line.strip() == "Ravi"</code>
<code>f.write(name)</code>	everything on one line	<code>f.write(name + "\\n")</code>
<code>int(row[2])</code> forgotten	<code>"88" + 1</code> is a <code>TypeError</code>	cast every CSV value
<code>line.split(",")</code>	breaks on quoted commas	<code>csv.reader(f)</code>
<code>open(csv)</code> without <code>newline=""</code>	blank rows between records	<code>newline=""</code>
reading twice in one with	second read gives <code>""</code>	store it, or reopen

Row 1 is the only one on this list that destroys data. The rest merely waste your afternoon. All eight are in today's practice notebook.

Putting it together — CSV in, report out

```
import csv
from collections import defaultdict

def load_marks(path):
    """Read the CSV into a list of clean dictionaries."""
    rows = []
    with open(path, newline="", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            row["mark"] = int(row["mark"])    # cast it!
            rows.append(row)
    return rows

def summarise(rows):
    """Average mark per subject."""
    totals = defaultdict(list)
    for r in rows:
        totals[r["subject"]].append(r["mark"])
    return {s: sum(m)/len(m) for s, m in totals.items()}

def write_report(summary, path):
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["subject", "average"])
        w.writerows([[s, f"{a:.1f}"] for s, a in summary.items()])

rows = load_marks("marks.csv")
write_report(summarise(rows), "report.csv")    # NEW filename
```



DictReader

header becomes keys



Cast on load

"88" becomes 88 at the door



with, twice

both files close themselves



Day 5's grouping

defaultdict by subject



Day 6 comprehension

builds the summary dict



One job each

load • summarise • write



A NEW output file

the input is never touched

Cast at the door, write to a new file, one job per function. In two weeks pandas does all of this in three lines — and you will know what it is doing.

Practice session — 30 minutes



Predict the output

Modes, newlines, exhausted files

8 questions



Spot & fix the bug

Wrong mode, no strip, split on CSV

4 questions



Write the code

Read, count, filter, write, append

5 tasks

Your notebook

[Day9_Practice_Questions.ipynb](#)

- The notebook makes its own files
- Nothing of yours is at risk
- Raise a hand — I am walking around
- Solutions shared after class

Before every open(), say the mode out loud. “w” means erase. If you did not mean erase, you wanted “r” or “a”.

TODAY IN TWELVE LINES

Recap

- 1 Always use `with open(...)` as `f:` — it closes the file for you.
- 2 Pass `encoding="utf-8"` on every text file.
- 3 `"r"` reads · `"w"` ERASES · `"a"` appends · `"x"` refuses to overwrite.
- 4 `for line in f:` is the default way to read — any file size.
- 5 Every line keeps its `\n`. Use `.strip()` before comparing.
- 6 `f.write()` adds no newline. `print(..., file=f)` does.
- 7 `line.split(",")` breaks on commas inside quoted fields.
- 8 Use `csv.reader` for lists, `csv.DictReader` for dicts.
- 9 Open CSVs with `newline=""` to avoid blank rows.
- 10 Every CSV value arrives as a `STRING`. Cast it at the door.
- 11 Wrap opens in `try/except FileNotFoundError`.
- 12 Write results to a `NEW` file — never over your input.

Homework

Write a word-frequency counter that reads a text file · Read a CSV and write out only the rows that pass a filter · Add `try/except` to every file function you have written

Next class — Topic 1.10

Exception handling & modules: `try / except / finally`, raising and writing your own exceptions, importing modules and packages